



WHEATON  
COLLEGE

*For Christ & His Kingdom*

# AI for Theological Inquiry

Lecture 4 — Sources & the theological corpus: *what*  
you ground on

Huihuo Zheng  
Wheaton College  
September 22, 2026

[huihuo.zheng@wheaton.edu](mailto:huihuo.zheng@wheaton.edu)

# Where we are: last week, *why* grounding matters

## Last week

- Wrapped the agent loop back on the raw model
- Vibe-coded a tool — and **baked a fabricated verse into code that ran green**
- The rule: *ground it, prove it, disclose it*

You saw *why* you must ground.

## This week

- Grounding means retrieving from **real sources**
- So: **which** sources? **whose** text?
- Assemble the **term corpus** — the texts you ground on all term

You choose *what* to ground on.

First, though — a proper look at the thing you actually did last week: *vibe coding*.

## Recap 1/5 — “Vibe coding”: what the term actually means

“There’s a new kind of coding I call ‘vibe coding’, where you fully give in to the vibes, embrace exponentials, and **forget that the code even exists** ... I ‘Accept All’ always, I don’t read the diffs anymore.”

— Andrej Karpathy, February 2025

### ↔ The term has already drifted

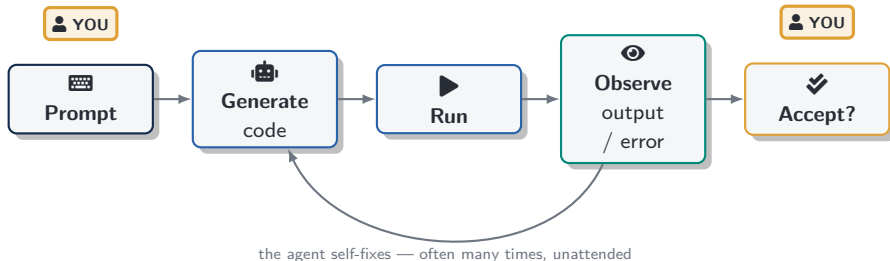
In everyday use “vibe coding” has come to mean *any* AI-assisted coding. That is not what it named. The original sense is narrower — and much sharper.

### 🚫 The load-bearing word is *not*

Not the model. Not the tool. Not the speed. **You do not read the code.** Everything else about vibe coding follows from that one omission.

So the definition is not about what the machine did. It is about *what you chose not to look at.*

## Recap 2/5 — The loop, and where you sit in it



### You touch the loop exactly twice

You write the **prompt** and you make the **accept decision**. Everything between is the agent. Vibe coding is what happens when that second touch becomes automatic.

### Why the loop is the whole story

An LLM answers *once*; an agent **runs the loop** until something looks done. “Looks done” is measured by the *program running* — never by the program being *right*.

## Recap 3/5 — It is a dial, not a switch

|                                                                                   | Level             | What you actually read   | Honest use           |
|-----------------------------------------------------------------------------------|-------------------|--------------------------|----------------------|
|  | L0 · Autocomplete | Every line; you wrote it | Anything             |
|  | L1 · Assisted     | Every generated line     | Anything             |
|  | L2 · Delegated    | The diff and the tests   | Work you will defend |
|  | L3 · <b>Vibe</b>  | Only that it <i>ran</i>  | Prototypes only      |
|  | L4 · Autonomous   | Nothing at all           | <i>Not research</i>  |

💡 Last week you ran at **L3** on purpose — that is why the fabrication got through.

**No rung is a sin; landing on one by accident is.** Pick the rung before you prompt — and name it when you disclose.

## Recap 4/5 — Five ways vibes fail (one matters most here)

### 1. Fabrication — the one that bites *this* course

Invented facts hard-coded **as data** — last week's wrong Philemon 1:6, in a script that **ran green**.

- 2. **Plausible-but-wrong logic** — it runs; the semantics are off. A stop-word list that drops *"not"*.
- 3. **Silent scope creep** — it rewrote a file you never mentioned.
- 4. **Dependency and licence debt** — packages and provenance you did not choose.
- 5. **Comprehension debt** — you cannot debug, defend, or **disclose** what you never read.

Code *launders* a claim: in a chat window it looks like a guess — in a script it looks like a result.

## Recap 5/5 — The discipline, and today's version of it

**Vibe to explore. Verify to publish.**

### Four moves that turn L3 into L2

1. **Read** what it wrote — at minimum the diff
2. **Ground** every factual claim in a real source  
(RAG — Wk 4)
3. **Test** it against something you can check  
(Eval — Wk 6)
4. **Disclose** model, tool, and *which rung* you were on

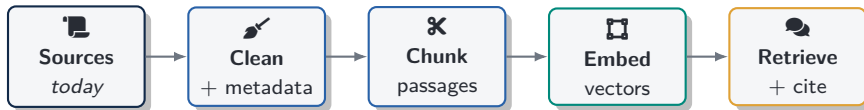
### The same rule in new clothes

In this afternoon's lab the agent will happily fetch and clean your texts — fast, and worth using. But *it* cannot tell you which canon, which translation, or which licence it just handed you.

**Vibe to *fetch*. Verify to *keep*.**

**Next week we build the retriever. This week we decide what it is allowed to find.**

# Everything downstream inherits what you choose here



## ← Garbage in → grounded in garbage

A perfect retriever over the **wrong edition** cites the wrong text — *confidently, with a page number*. The corpus is the ceiling on everything the agent can honestly say.

## ! The data is not neutral

Which books, which translation, which manuscript, which notes — each is a **decision** someone already made. You inherit it, so you must **see it**.

# Where digital primary sources live (public-domain first)

## Scripture (openly licensed)

**World English Bible** (public domain), **KJV**, ASV, Darby, Young's Literal — via bible-api.com, STEP Bible, or plain text.

## Classics & the Fathers

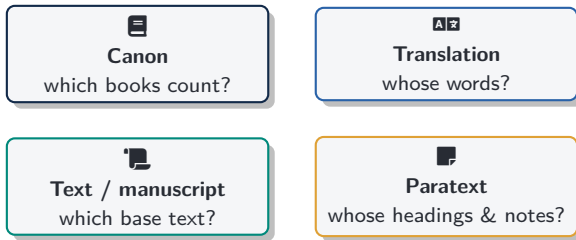
**CCEL** (ccel.org) — Augustine, Aquinas, Calvin, Luther; **Project Gutenberg**; **Sacred Texts** across traditions.

## Originals & critical text

**Perseus** (Greek/Latin), SBLGNT, open Hebrew (OSHB) — when the *original language* is the object of study.

**i** **Prefer public-domain / open-licensed sources.** They are reproducible, redistributable, and *disclosable* — the same values that made us run on an open ALCF model. Modern copyrighted translations come with strings (next-but-one slide).

# The heart of today: “the data is theologically loaded”



**Four decisions are *already made* inside any text file you download.**

None is a bug to “fix.” Each is a scholarly choice — your job is to **name** it and **disclose** it.

# Canon & translation: the choice is theology, not plumbing

## Canon — which books?

Protestant **66** · Catholic **73** (Tobit, Judith, Wisdom, Sirach, 1–2 Maccabees...) · Orthodox more.

**Download “the Bible” and you have quietly chosen a canon.**

## Translation — whose words?

Isaiah 7:14: KJV/ESV “a **virgin** shall conceive” vs NRSV “a **young woman**” (Heb. *almah*).

**Same verse, different doctrine implied — by the translator.**

**?** Ask your RAG agent “*what does the Bible say about X?*” — and it answers from **the canon and translation you loaded**, with no idea it made a choice. **You** have to know which Bible is in the box.

## Text & paratext: the words that may not be original



### Manuscript / base text

*Textus Receptus* (KJV) vs the **critical text** (NA28). Disputed passages: the **longer ending of Mark** (16:9–20), the *pericope adulterae* (John 7:53–8:11), the *Comma Johanneum* (1 John 5:7).

Present, bracketed, or footnoted — depending on your file.



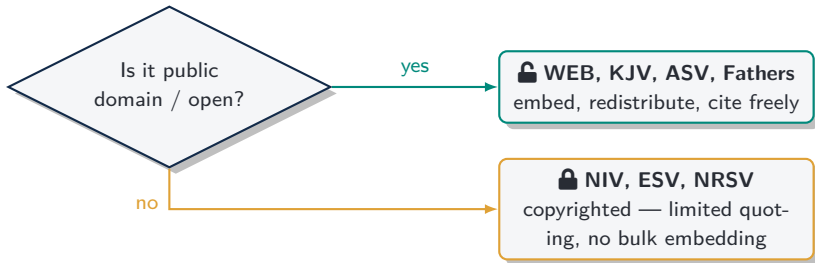
### Paratext — the editor's hand

Chapter & verse numbers (**medieval**, not original), section headings, cross-refs, study notes — all **added**.

**Chunk by “verse” and you have adopted a 13th-c. editor's boundaries as your unit of meaning.**

How you *chunk* (next week) is itself interpretive — verse, pericope, or paragraph?

# Copyright: why we ground on public-domain translations



## ✓ The default for this course

Build your vector DB on **open** texts. A copyrighted translation baked into a redistributable index is a **licensing problem**, not just a scholarly one.

## ⚖️ Need a modern translation?

Quote **short excerpts** under fair use, keep it **out of the shared index**, and **disclose** the edition. When in doubt, ask.

# From texts to a corpus: document every source



## A “source card” per text

- **Work & author** — e.g. Luther, *95 Theses*
- **Edition / translation** — who, what year
- **Where from** — URL + date accessed
- **License** — public domain / CC / ...
- **Canon / base text** — if scripture
- **Known gaps** — what it is *not*



This *is* the disclosure appendix — applied to your **data**, before a single query. Provenance you write **now** is provenance you don't have to reconstruct in Week 14.

A corpus is not a folder of files — it is files *plus* a record of what they are.

# Hands-on: assemble *your* term corpus (~40 min)

## Steps

1. Start from your **interest statement** (Wk 1) — what texts must the agent see?
2. Find **3–6 sources**, public-domain first
3. Download to corpus/ as clean .txt
4. Write a **source card** for each
5. Note one **loaded decision** you inherited

## Let the agent help — then check

opcode can **download & clean** (like the Luther capstone). But **you** verify the edition and license — vibe to fetch, **verify to keep**.

## Start small

A **tight, well-documented** corpus beats a huge, murky one. You can grow it toward the Week 7 proposal.

# What is *not* in your corpus is a claim too

## Silent exclusions

One tradition's commentators? One century?  
One language? English only?

The agent will speak **fluently** about what it *can-not* see — and never say so.

## Name the boundary

Write your corpus's **scope & limits** into the record:  
whose voices are in, whose are out, and **why**.

That honesty is graded — and it is good scholarship.

 **Bias enters at the corpus**, long before the model. We return to bias-aware evaluation in Week 10 — but it starts with the folder you build today.

# Project touchpoint & assignment

## Touchpoint — corpus → proposal

Your term corpus is the **evidence base** for the Week 7 proposal. If you can't find sources for your question, the question needs reshaping — **better to learn that now.**

## Due next week

- A corpus/ of **3–6 public-domain sources** + a **source card** for each
- **One paragraph:** a loaded decision you inherited, and your corpus's scope & limits

## Reading

Claire Clivaz on digital humanities & biblical studies (selection).

**Next week: RAG — retrieve → augment → generate over the corpus you just built.**